# ACUBOT Final Report

**Course:** CSE 322 - Cloud Computing

**Project:** ACU AI Chatbot (aka ACUBOT)

**Team:** Beyza Yüz (231401701), Ceren Yurdunol (231401049), Armita Tavakolshoar (241401906)

**Repository:** https://github.com/beyz100/ACUBOT

## 1. Introduction

ACUBOT is a chat application that answers questions about Acıbadem University in Turkish or English, using only data collected from the university's public websites which include information that is normally scattered across the main university site and the Bologna Information System (faculty descriptions, department lists, course catalogues, contact information, admission rules, etc.) is collected into a single natural-language interface. Our main objectives **with this project** were to build a Django web application, scrape public university data, run an open-source LLM entirely locally via Docker, and ensure accurate, grounded answers through prompt engineering—all starting with a single docker compose up command**.**

## 2. Architecture

The system uses five Docker containers connected via the acubot_default bridge network.

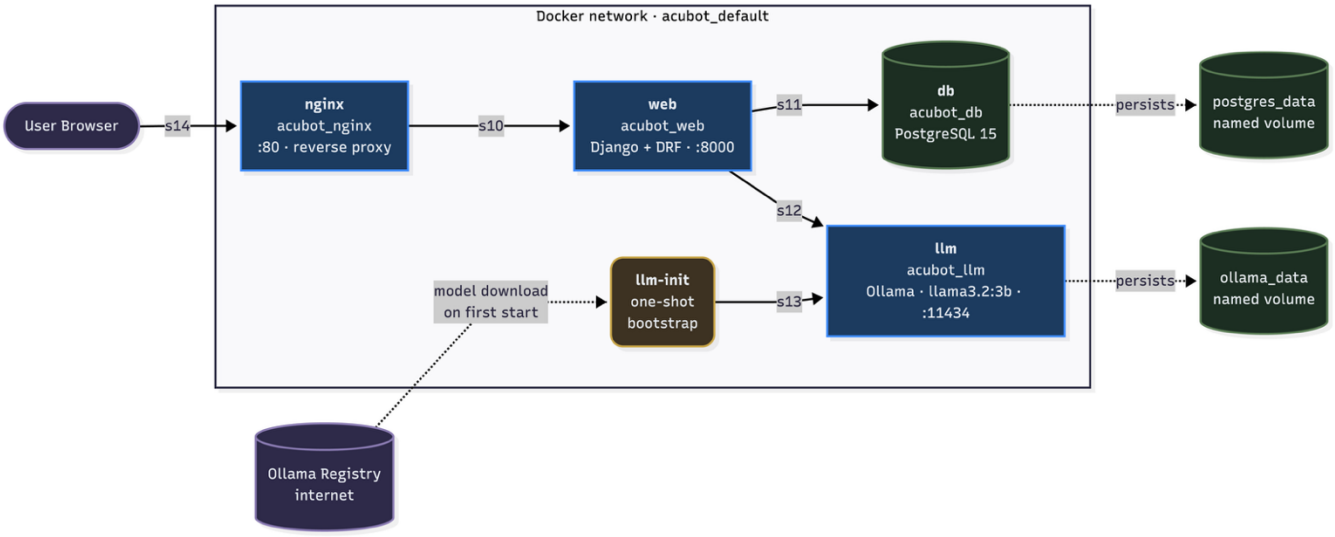


**Figure 1.** Diagram showing the overall project architecture.

- **nginx (acubot_nginx)** listens on port 80. The streaming endpoint /api/chat/ask/ has bufferin g disabled, otherwise the LLM's token-by-token output would be held back and shown to th e user at once.
- **web (acubot_web)** includes Django + Django REST Framework. Serves the /chat/ interface and the REST API. On startup, entrypoint.sh waits for the database, applies migrations, runs the seed script, and creates the default superuser.
- **db (acubot_db)** is made up of PostgreSQL 15. Holds both the structured knowledge base (faculty/department/course tables) and the chat history (Conversation/ChatMessage). The pg_trgm and full-text-search extensions are enabled.
- **llm (acubot_llm)** is Ollama, serving the llama3.2:3b model. The model size was chosen so that the whole stack still starts on a 16 GB-RAM student laptop using only the CPU.
- **llm-init (acubot_llm_init)** is a bootstrap container that connects to Ollama, checks whether the configured model is already cached, pulls it if not, and exits cleanly. Compose's service_completed_successfully condition then unblocks web.

## 3. Implementation

- **Web Framework (Django 5.x & Django REST Framework):** Django's built-in ORM, admin panel, and DRF serializers provided a quick start for the required assignment features, such as the REST API and chat history.

- **Database (PostgreSQL 15):** Instead of adding a complex vector database, PostgreSQL's native SearchVector (for weighted keyword search) and TrigramSimilarity (for typo tolerance) were used.

- **LLM Serving (Ollama):** Ollama was utilized to run the language model locally. It was preferred because it exposed a simple HTTP API and was easily integrated with Docker Compose. The setup process was much lighter and faster compared to alternatives like vLLM or TGI.

- **Language Model (Llama 3.2 - 3B):** The 3-billion-parameter Llama 3.2 model was chosen for the chatbot engine. During testing, it was observed that 7B-8B models caused heavy latency on CPU execution, while 1B models struggled significantly with Turkish grammar. The 3B version was selected as the optimal balance between response time and language accuracy.

- **Reverse Proxy (Nginx Alpine):** Nginx was configured as the reverse proxy for the application to handle streaming-friendly buffering for the /api/chat/ask/ endpoint and to serve static files efficiently within the Docker network.

- **Scraping Tools (Selenium & BeautifulSoup4):** A hybrid scraping approach was taken for data extraction. Selenium was used for the Bologna system since its pages are heavily JavaScript-driven and contain multiple iframes. For the main university website, which is mostly rendered by the server, the standard requests library and BeautifulSoup4 were utilized.

**Data & Retrieval Layer Architecture:**

The data layer was structured around four core models: Faculty, Department, Course, and a free-form UniversityInfo table for schema-less data. Additionally, the chat layer was designed using session Conversation and ChatMessage models. Database seeding was managed through seed.py, utilizing Django's update_or_create method to ensure that duplicate rows were not created during multiple executions.

For the retrieval layer, several advanced PostgreSQL techniques were implemented:

- **Weighted Full-Text Search:** SearchVector was configured to perform a weighted search, giving higher priority to primary information such as course codes and names.
- **Similarity Matching:** Trigram similarity (TrigramSimilarity) was integrated to enable spelling-tolerant database queries.
- **Synonym Mapping:** A bidirectional Turkish-English synonym map was applied to expand search queries effectively.
- **Intent-Based Routing:** Intent-based category routing was utilized to direct specific free-form keywords directly to their corresponding data categories.
- **Safety Fallback:** A fallback mechanism was established to prevent empty contexts. If a topical filter yielded no results after matching a department, the system defaulted to returning the full catalog of that department.

## 4. Docker Setup

- **Dockerfile Setup:** The web service Dockerfile was based on the python:3.12-slim image. System-level dependencies such as libpq-dev (PostgreSQL client), netcat-openbsd (for startup synchronization), and chromium with xvfb (for headless Selenium scraping) were installed. Python dependencies were subsequently installed from requirements.txt.
- **Docker Compose Specifications:** Several architectural configurations were implemented in docker-compose.yml:
    - **Ordered Startup:** The db service was configured with a pg_isready health check, and the web service was set to wait for a service_healthy condition. Additionally, an llm-init service was introduced to download the model and exit, while the web service waited for its service_completed_successfully status.

- **Environment Configuration:** Dynamic configuration was enabled using the ${VAR:-default} pattern for database credentials, model names, and superuser details, allowing easy overrides via a .env file.
- **DNS Hardening:** To resolve occasional Docker Desktop DNS resolution failures on macOS (which caused "network unreachable" errors during ollama pulls), external DNS servers (1.1.1.1, 8.8.8.8) were explicitly pinned to the llm service as a fallback.
- **Bind Mounts:** The local directory was mounted to /:app in the web service to enable instant code reloading via Django's StatReloader during development.

## 5. AI Integration

The primary objective of the AI integration was to ensure accurate, domain-specific responses strictly bound by the university's data, preventing any model hallucination.

- **Model Selection:** llama3.2:3b was selected as the final inference model. During initial testing, qwen2.5:3b exhibited grammatical inconsistencies in Turkish, while llama3.1:8b and Qwen2.5 7B introduced unacceptable CPU latency (>2 minutes per response). The 3-billion-parameter Llama 3.2 provided the optimal balance, delivering coherent Turkish responses within around 20 seconds while operating efficiently within a 16 GB RAM Docker environment.

- **Prompt Engineering:**
  - Strict adherence to the provided knowledge base.
  - Exact string matching for faculty, department, and course names without abbreviation.
  - Telling of missing information rather than generating approximations or placeholders.
  - Prohibition of conversational prefixes (e.g., "User:", "Assistant:"). A dedicated _scrub_labels helper function was implemented to prevent any leakage of these prefixes.

- **Generation Parameters:** Inference parameters were strictly defined to prioritize determinism and factual accuracy: temperature=0.0, top_p=0.9, num_ctx=4096, and num_predict=2048.

## 6. Evaluation

**Question 1:** "Acıbadem Üniversitesi telefon numarası nedir?"
**Response:** "Acıbadem Üniversitesi telefon numarası +90 216 500 44 44'dir."
**Correct?:** ✓

**Notes:** Intent → category='contact' short-circuit at work.

**Question 2:** “What is the address of Acıbadem University?”
**Response:** “Kerem Aydınlar Kampüsü, Kayışdağı Cad. No:32, 34752 Ataşehir / İstanbul”
**Correct?:** ✓
**Notes:** Language detection correctly classified the query as English.

**Question 3:** “Hangi fakülteler var?”
**Response:** All five faculties (Tıp, Eczacılık, Sağlık Bilimleri, Mühendislik ve Doğa Bilimleri, İnsan ve Toplum Bilimleri) listed in full and correctly.
**Correct?:** ✓
**Notes:** “List all” intent triggers the full Faculty table being injected into context.

**Question 4:** “What faculties does Acıbadem University have?”
**Response:** The same five faculties, this time with their English names alongside (Faculty of Pharmacy, School of Medicine, …).
**Correct?:** ✓
**Notes:** Same retrieval path; only the formatting differs because of the detected language.

**Question 5:** “Bilgisayar Mühendisliği bölümündeki dersleri listele”
**Response:** All 46 courses are listed in order with their codes (CSE 101 Programlamaya Giriş, MAT 111 Kalkülüs I, …, CSE 322 Bulut Bilişim, …).
**Correct?:** ✓
**Notes:** Filler tokens like “bölümündeki” and “tüm” are not treated as topical, so a department match plus filler-only residual returns the whole catalogue.

**Question 6: “**Which departments are there in Acıbadem University?”
**Response:** 11 departments, alphabetical, each labelled with its faculty (Bilgisayar Mühendisliği, Eczacılık, Hemşirelik, Psikoloji, Sosyoloji, …).
**Correct?:** ✓
**Notes:** No duplicates, name fidelity preserved.

**Question 7:** “CSE 322 dersi nedir, kaç ECTS?”
**Response:** “CSE 322 — Bulut Bilişim, 6 ECTS, Bilgisayar Mühendisliği bölümü”
**Correct?:** ✓
**Notes:** Trigram + full-text combined; both code and name are indexed.

**Question 8:** "Yapay zeka ile ilgili ders var mı? Varsa kodu ne?"
**Response:** "Evet, CSE 332 Veri Bilimi ve Yapay Zeka dersi mevcuttur (6 ECTS).

**Correct?:** ✓
**Notes:** Full-text search successfully matched the keyword "Yapay zeka" to the relevant course code and title.

**Question 9:** “Web programlama dersleri”
**Response:** CSE 220 Web Programlama (Bilgisayar Mühendisliği, 6 ECTS) plus two related courses.
**Correct?:** ✓
**Notes:** The topical filter is tested here.

**Question 10:** “ASEV nedir?”
**Response:** “Acıbadem Sağlık ve Eğitim Vakfı (ASEV) tarafından 2007 yılında kurulmuştur.”
**Correct?:** ✓
**Notes:** Pulled from UniversityInfo with category='academic'.

**Accuracy Assessment:** During sample evaluations, the model successfully maintained language alignment and generated responses strictly confined to the provided knowledge base. Initial instances of hallucination were identified as symptoms of a truncated 2048-token context window. This issue was entirely resolved by expanding the parameter to 4096 tokens.

**Latency Profile:** Cold-start inferences (when the model is not yet cached in RAM) were observed to take 30–50 seconds. Once initialized, standard warm-start responses were processed within 8–20 seconds. Extensive data queries extended latency up to 30 seconds due to high output token volume. This latency was accepted as a necessary trade-off, as deploying a faster 1B parameter model resulted in severe degradation of linguistic quality.

**Known Limitations:** Currently, extreme abbreviations (e.g., "BM" for "Bilgisayar Mühendisliği") fail to resolve, as they do not meet the minimum trigram similarity threshold. Furthermore, inquiries regarding academic staff are intentionally unhandled; since staff directories were excluded from the knowledge base, the system prompt was strictly configured to output a "data unavailable" response rather than generating speculative answers.

## 7. Challenges

- **Context-Window Truncation:** Reducing the num_ctx parameter from 4096 to 2048 to speed up inference caused silent prompt truncation. As a result, the model hallucinated fake data. Log analysis showed retrieval was correct, isolating the issue to the LLM context window. The parameter was restored to 4096, and the hallucinations were eliminated.

- **Cross-OS Line Ending Conflicts:** Container crashes (exec /app/entrypoint.sh: no such file or directory) occurred on Windows host machines due to CRLF line endings breaking the script's shebang. This cross-platform issue was fixed by adding a .gitattributes file (*.sh text eol=lf) to enforce LF endings across all environments.
- **Containerized Selenium Execution:** Running Selenium inside a headless Docker container required installing chromium and chromium-driver packages and using specific flags (--no-sandbox, --disable-dev-shm-usage). A fallback using webdriver-manager was also implemented for host machines without native drivers.
- **Persistent Database Artifacts:** Old test data persisted in the database volume because Django's update_or_create does not delete removed entries. This was resolved by manually clearing the Docker volume (docker compose down -v) to ensure a clean database state.
- **Future Architectural Considerations:** Relying purely on PostgreSQL full-text search limited flexibility. Integrating a vector store (like pgvector or ChromaDB) would have provided better semantic retrieval. Additionally, the manual scraping process could be converted into an automated Celery task for semesterly updates.

## 8. Team Contributions

- **Beyza Yüz [beyz100] (Backend / AI / DevOps):** The Docker Compose infrastructure, continuous integration (CI) workflows, and Nginx reverse proxy configurations were established. The core AI integration (Ollama), system prompt engineering, and custom language-detection utilities were implemented. Furthermore, the hybrid retrieval architecture (incorporating trigram, full-text, and intent-based searches) was designed.
- **Ceren Yurdunol [Mirket07] (Backend / REST API):** The session-scoped database models (Conversation, ChatMessage) and Django REST Framework serializers were developed. The core REST API endpoints were built and wired to the frontend chat-history UI. Additionally, streaming response handling was successfully integrated into the system, and Docker container compatibility for the automated web scrapers was ensured.
- **Armita Tavakolshoar [armita-41] (Data / Testing / Frontend):** The dual-scraper data extraction system (Selenium for the dynamic Bologna system, requests + BeautifulSoup for the main site) was developed, aiding in the generation of the JSON datasets. The seed.py database script was rewritten to support new fixture schemas. Furthermore, the initial chatbot user interface was designed, and LLM performance testing scripts were created and executed.

  **Note:** Commit distributions and individual contributions were documented via GitHub Insights. A structured "develop-then-merge-into-main" workflow was systematically utilized to maintain a meaningful Git history throughout the project lifecycle.


**Figure 2.** Graph showing the commits over time by contributors


**Figure 3.** Graphs showing the contributions of each contributor.


**Figure 4.** Code frequency graph showing additions and deletions made per week.